

江南大学

22 学年—23 学年第 2 学期 算法设计与实践实验报告

专业名称： 数字媒体技术 实验学时： 4

课程名称： 算法设计与实践 任课教师： 晏涛

实验题目： 动态规划算法用于数字图像处理（Seam Carving）

实验环境： PyCharm Community Edition Python3.10

实验目的：

将动态规划算法应用于数字图像处理中的 Seam Carving（缝隙裁剪）任务。Seam Carving 是一种图像内容感知的图像缩放方法，它能够在保持图像主要内容的前提下，自动调整图像的尺寸。

实验内容：

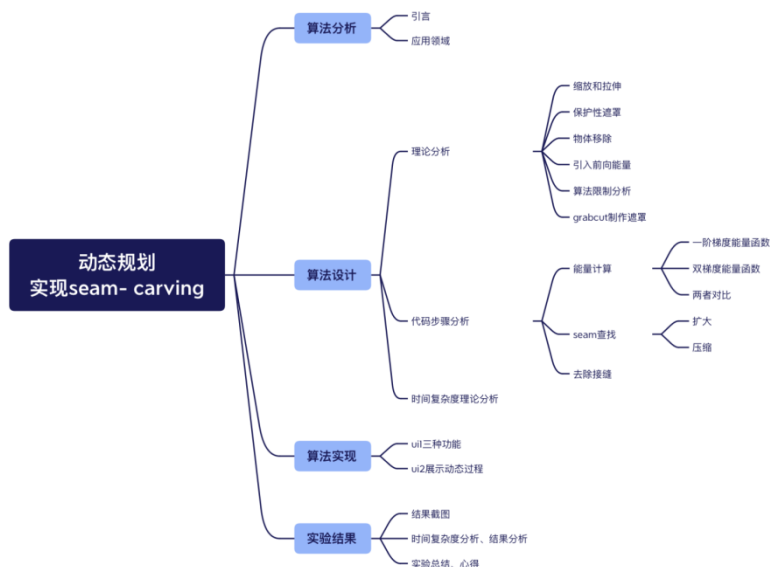
通过使用动态规划算法进行 Seam Carving 处理，可以实现图像的内容感知缩放。相比传统的缩放方法，Seam Carving 可以在不丢失图像重要内容的情况下进行尺寸调整，从而更好地适应不同显示屏幕或应用场景的需求。实验结果将以 gui 界面的方式展示经过 Seam Carving 处理后的图像，可以观察到图像的尺寸变化以及保留的主要内容。

实验要求：

将动态规划算法应用于数字图像处理中的 Seam Carving（缝隙裁剪）任务。

问题分析：

一、实验流程



二、算法分析

2.1 引言

常用的图像处理和调整技术有很多，可以实现图像大小变化的常见算法有 Scale（缩放）、Seam Carving（缝合裁剪）、Crop（裁剪）。



1、**Scale（缩放）**：图像缩放是改变图像的尺寸大小的过程。它可以将图像的尺寸增大或减小，通过调整像素的分辨率来改变图像的大小。缩放通常可以通过插值算法来实现，常见的方法包括最近邻插值、双线性插值和双三次插值等。

2、**Seam Carving（缝合裁剪）**：Seam Carving 算法是一种用于图像内容保持的调整方法。它通过删除图像中能量值最低的路径（也称为"缝"），来调整图像的大小。Seam Carving 可以在保持图像的主要特征的同时，尽可能地减少对重要内容的影响。它通常使用动态规划来确定最佳路径，并通过逐步删除路径上的像素来调整图像。

3、**Crop（裁剪）**：图像裁剪是通过剪切图像的一部分来改变图像的大小和内容。裁剪通常是根据用户指定的感兴趣区域或特定的尺寸要求来进行的。通过裁剪，可以去除图像中不需要的部分或调整图像的纵横比例。

2.2 应用领域

总的来说，Seam Carving 算法在以下领域有广泛的应用：

1、**图像缩放和裁剪**：Seam Carving 可以用于自适应性图像缩放和裁剪，使得图像的尺寸可以按照用户需求进行调整，而不会导致重要内容的失真。相比传统的等比例缩放或裁剪，Seam Carving 能够更好地保持图像的结构和内容完整性。

2、**图像内容保护**：Seam Carving 可以用于在调整图像尺寸时保护感兴趣的目标或重要区域。通过指定目标的保护区域，算法会在删除 Seam 时尽量避免影响到这些区域，从而保持目标的完整性。

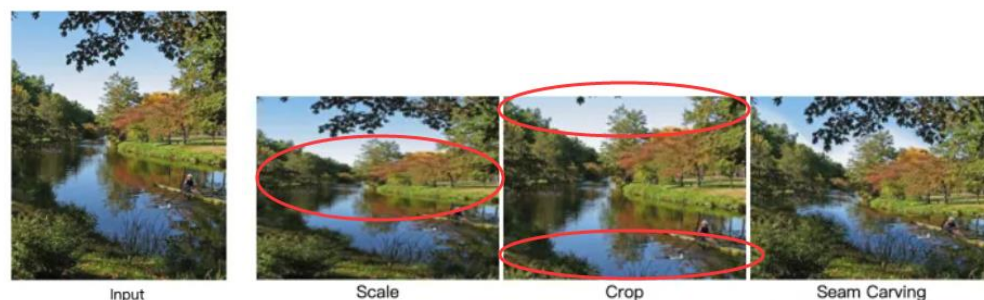
3、**图像压缩**：Seam Carving 可以作为一种图像压缩的方法，相比传统的压缩算法，Seam Carving 能够更有效地减小图像的尺寸，同时保持图像的视觉质量和内容完整性。

4、**图像扩展**：Seam Carving 不仅可以缩小图像，还可以扩展图像。通过插入 Seam，可以在图像中增加额外的像素，从而增加图像的宽度或高度。这对于需要调整图像尺寸或在特定应用中需要更多像素的情况非常有用。

5、**图像对象移除**：Seam Carving 可以用于移除图像中的不需要的对象或区域。通过选择和删除包含对象的 Seam，可以在不留下明显痕迹的情况下移除图像中的对象。

如果我们想要实现的效果是调整图像的大小，同时尽可能地保留图像中的重要内容和结构，那么 Seam Carving 是其中最好的方法。

下面展示了以上提到的对输入图像的尺寸进行改变的几种方法的比较，可以看到 Scale 方法会导致内容变形，Crop 方法虽然保持了关键内容的形状，但却丢失了信息，只有 Seam Carving 既保持了原始场景的所有内容，同时又不会让内容产生大的形变。



（观察图像可知：

Seam Carving:相比其他图像缩小方法保留了更多的信息，且维持了主体的比例。

Scale 方法会导致目标变形。

Crop 方法丢失了图像两边边界区域的内容。）

此外，这个算法不仅可以缩小图像，还可以扩展图像，产生新的内容，却又看起来非常和谐。

找个缝，内容一般



找到前 k 个最佳接缝，然后对内容进行平均





当我们用适当的方式使用它时，它可以用于去除场景中我们不需要的目标。用户需要标注需要保留的物体（红色掩模）和需要去除的区域（绿色掩模）

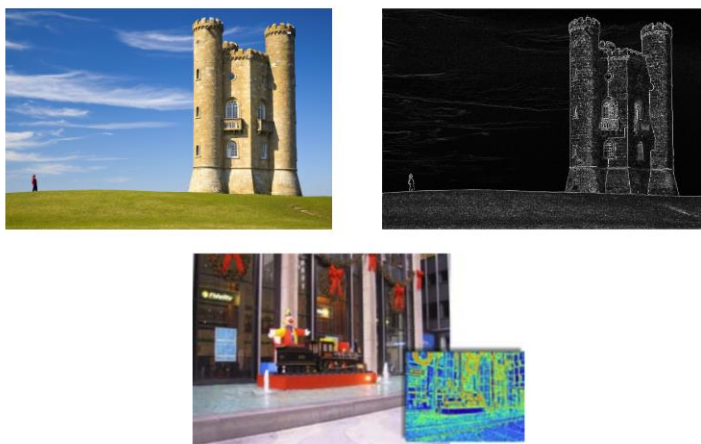
三、算法设计

3.1 理论分析

我们首先需要有一个衡量图像中哪些区域不重要的度量标准。能量函数的目的是为每个像素分配一个值，表示其对图像的重要性。那些与周围环境融合并且不引人注目的像素应该具有较低的能量值。原始 **Seam Carving** 论文中提出的能量函数是简单的梯度幅值函数：

$$e_I = \left| \frac{\partial}{\partial x} I \right| + \left| \frac{\partial}{\partial y} I \right|$$

下面我们可以看到一张图像及其“能量图”，能量图是通过简单的卷积操作计算得出的梯度幅值。



示例图像和能量图

在实际操作中我们还会看到更复杂的能量图，但目前梯度幅值可以给出满意的结果。

将垂直 **seam** 定义为图像中从顶部到底部的 8 连通像素路径，每行恰好包含一个像素。同样地，水平 **seam** 是从右到左的路径，每列恰好包含一个像素。移除一个 **seam** 的效果是将像素向上（或

向左) 移动, 以补偿缺失的路径。通过反复删除或插入 seam, 我们可以将图像重新调整为新的尺寸。

能量图帮助我们确定要移除的 seam。将 seam 的代价定义为沿着 seam 上的像素的能量值之和, 我们寻找最小化这个 seam 代价的最优 seam。

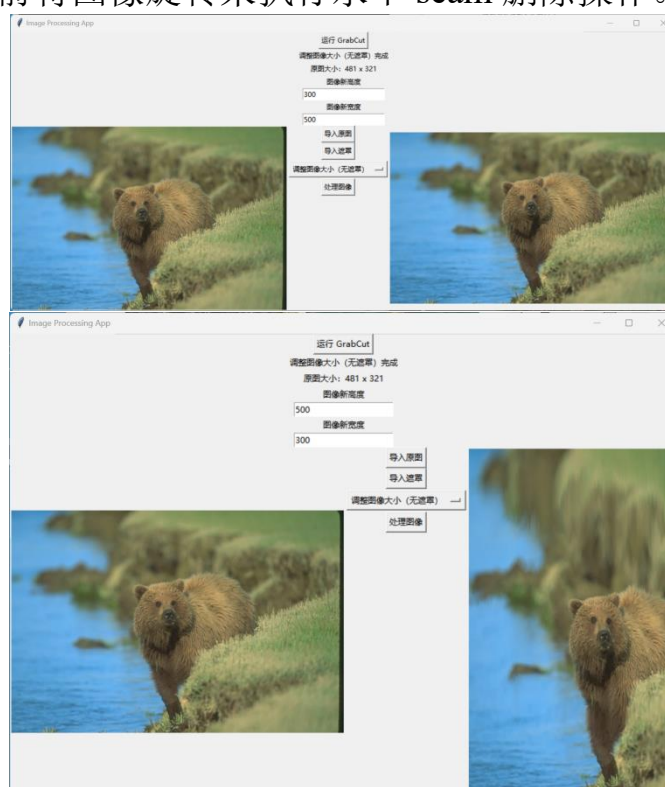
找到最优 seam 是一个简单的动态规划问题。第一步是从第二行遍历到最后一行, 并计算每个条目 (i, j) 的所有可能连通 seam 的累计最小能量 M :

$$M(i, j) = e(i, j) + \min(M(i-1, j-1), M(i-1, j), M(i-1, j+1))$$

填充 M 后, 最后一行的最小值将指示最小连通垂直 seam 的结束位置。然后, 我们可以从 M 的最小条目回溯, 找到最优 seam 的路径。

3.1.1 缩放和拉伸

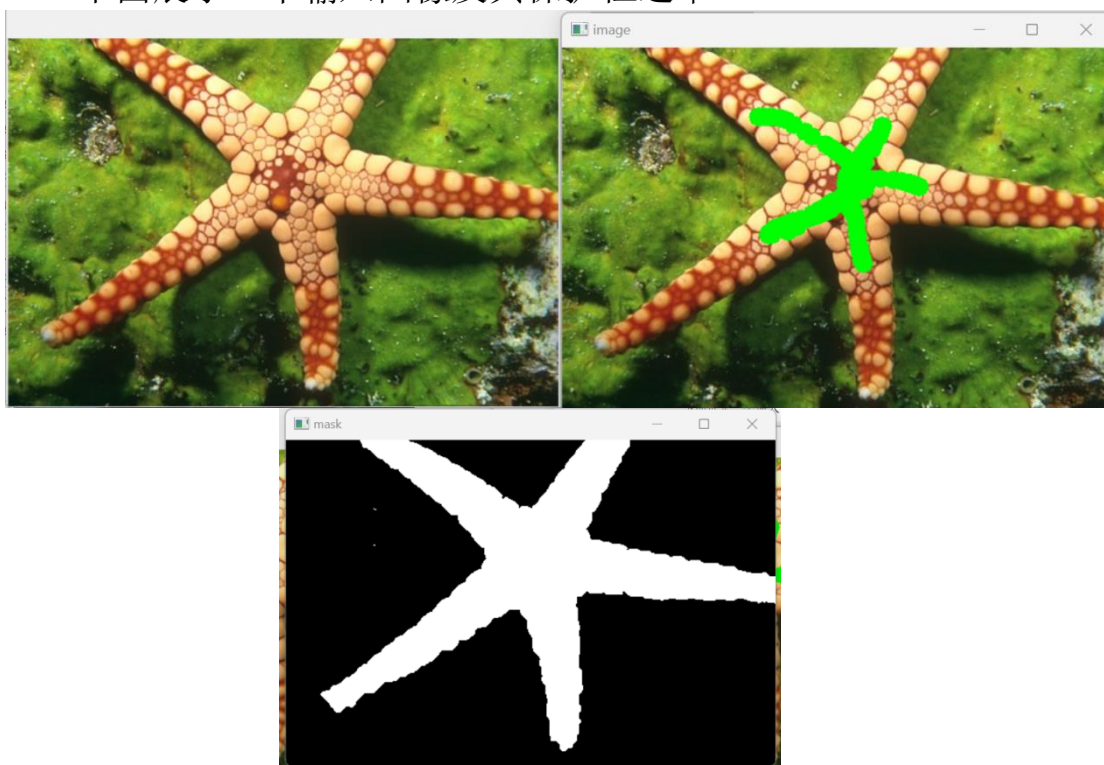
运行垂直 seam 删除来将图像宽度减小一半, 也可以通过在应用算法之前将图像旋转来执行水平 seam 删除操作。



3.1.2 保护性遮罩

有时我们希望指定图像中不应该被修改的区域。这可以通过向删除算法提供二进制遮罩来实现。在相应的能量图中, 可以为遮罩的白色区域提供较大的正能量值, 以强烈阻止最优 seam 通过该区域。

下面展示一个输入图像及其保护性遮罩。



在 seam 删除过程中,受保护的区域将保持完好。作为实现细节,重要的是同时更新图像和遮罩的 seam 删除,以使它们保持一致。



我们还可以通过对删除过程进行简单修改来执行 seam 插入操作。我们按照原始论文中描述的技术进行操作: 要将图像扩大 k 倍, 我们找到前 k 个要删除的 seam。然后, 我们按照相反的顺序将它们添加回图像中, 通过对 seam 上的像素值进行平均来进行插入。

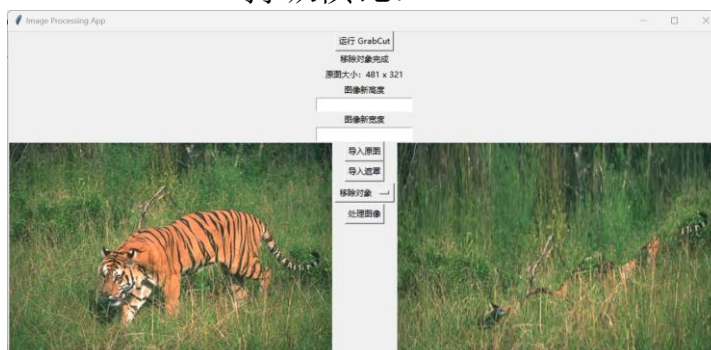
3.1.3 物体移除

我们可以使用之前的相同遮罩技术来选择要删除的区域; 与其在相应的能量图区域中提供较大的正能量, 我们可以给该区域提供较大的负能量。通过重复删除 seam 直到删除遮罩为空, 我们

可以从图像中移除对象。

我们可以同时使用对象删除遮罩和保护性遮罩。

在一般的实现上还会在对象删除后执行 **seam** 插入操作以保持纵横比：



通过在对象删除后执行 **seam** 插入操作，我们可以填补通过删除对象创建的空白部分，恢复图像的原始纵横比。这有助于避免失真，保持图像的视觉连贯性。**Seam** 插入通过复制沿着 **seam** 的相邻像素，向图像中添加新像素。

对象删除、**seam** 插入和保护性遮罩的组合可以实现精确和可控的图像编辑。它为从图像中删除不需要的对象并同时保持整体构图和视觉完整性提供了强大的工具。

在选择是执行垂直还是水平 **seam** 删除时，取决于图像的上下文，并作为程序的可选参数（默认情况下，删除垂直 **seam**）。

3.1.4 动态规划步骤梳理

1、计算能量图：首先，根据图像的像素值计算能量图，能量图表示每个像素的重要性或者梯度值。常用的计算能量图的方法是计算每个像素点的梯度值，例如使用 **Sobel** 算子或其他梯度算子。

2、初始化动态规划表格：创建一个与原始图像大小相同的动态规划表格，表格的每个元素表示从图像的顶部到该位置的路径的能量累积值。表格的初始值可以设为能量图的第一行。

3、动态规划计算：从第二行开始逐行计算动态规划表格的每个元素的值。对于每个元素，选择从上一行相邻的三个元素中能量累积值最小的路径，并将该路径的能量累积值加上当前位置的能量值，更新当前位置的动态规划表格值。重复这个过程，直到计算完最后一行的动态规划表格值。

4、最优路径回溯：在计算完动态规划表格后，可以通过回溯找到能量累积值最小的路径，即最优的拼缝。一般从最后一行的动态规划表格中找到能量累积值最小的元素，然后向上遍历表格，每次选择能量累积值最小的相邻元素作为路径的一部分，直到回

溯到第一行，得到最优路径。但需要注意，在 Seam Carving 中，最优路径的回溯过程通常涉及两个方向：从最后一行向上回溯，或从第一行向下回溯。具体的回溯方向取决于问题的具体实现和要求。

通常情况下，最优路径的回溯是从最后一行向上回溯的。这是因为在 Seam Carving 中，删除拼缝是最常见的操作，而删除操作会导致图像的高度减小。因此，从最后一行向上回溯可以找到能量累积值最小的路径，并将其删除，实现图像的纵向缩放。

然而，在某些特定情况下，也可以从第一行向下回溯。例如，在图像的顶部或底部需要插入拼缝时，可以从第一行向下回溯，选择能量累积值最小的路径，并将其复制并插入，实现图像的纵向放大。

总之，最优路径回溯的具体方向取决于要实现的操作（删除或插入）以及问题的要求。根据需要，可以选择从最后一行向上回溯或从第一行向下回溯。

5、删除或插入拼缝：根据最优路径，可以选择删除或插入拼缝。删除拼缝意味着在最优路径上移除对应的像素行，从而减小图像的宽度。插入拼缝意味着在最优路径上复制并插入对应的像素行，从而增加图像的宽度。重复删除或插入拼缝的过程，直到达到所需的图像宽度或高度。

3.3 时间复杂度理论分析

Seam carving 的时间复杂度主要取决于以下几个因素：

1、图像尺寸：时间复杂度与图像的宽度（W）和高度（H）相关。通常情况下，时间复杂度会随着图像尺寸的增加而增加。

2、能量计算：能量计算是 seam carving 算法中的一个重要步骤，用于确定每个像素的能量值。能量计算的时间复杂度通常与图像尺寸成正比，因为需要遍历每个像素并计算其能量。

3、动态规划：在 seam carving 中，动态规划用于查找最小能量路径。动态规划的时间复杂度与图像尺寸成正比，通常为 $O(WH)$ 。这是因为需要填充一个大小为 $W \times H$ 的能量矩阵，动态规划部分的代码通常使用双重循环遍历图像的每个像素，以填充能量矩阵并计算最小能量路径。因此，seam carving 的时间复杂度可以表示为 $O(WH + E)$ ，其中 E 是能量计算的时间复杂度。实际的时间复杂度还会受到算法优化、实现细节以及硬件性能的影响。对于大型图像，seam carving 可能需要较长的时间来执行，因此在实际应用中需要权衡时间和效果的平衡。

3.1.5 引入前向能量进行的改进

在进行 seam 删除时，我们会删除图像中的一条路径(seam)，然后将剩余的像素重新排列，以调整图像的大小。在这个过程中，原本相邻的像素可能会变得不再相邻，而新的相邻关系会引入一些额外的能量。

想象一下，当我们删除一条 seam 时，周围的像素可能会发生微小的移动和调整，这会产生一些新的边缘和细微的变化。这些新的边缘可能对图像的内容和结构产生影响，但传统的能量函数无法捕捉到这些变化。

前向能量是一种更加细致入微的能量计算方法，它考虑了这些新边缘的影响。它通过观察去除 seam 后的图像来计算能量，以反映像素重排引入的变化。这样可以更准确地衡量图像中各个像素的重要性，保持图像的结构和连续性。

前向能量的计算考虑了像素重排后的变化，以更准确地评估像素的重要性。在传统的梯度幅值能量函数中，能量的计算只基于原始图像的梯度信息，即像素值的变化情况。然而，当我们删除 seam 时，图像中的像素会重新排列，原本非相邻的像素可能会成为相邻的像素。这种像素的重新排列会引入新的边缘和细节，这些在原始能量函数中并未考虑。

前向能量的计算方式是基于以下观察：在删除一个 seam 后，新形成的边缘由先前非相邻的像素组成，它们现在成为相邻的像素。因此，我们可以通过计算这些新形成的边缘的能量来捕捉到像素重排引入的变化。

具体来说，计算前向能量的步骤如下：

- 1、根据去除 seam 后的结果图像，重新计算每个像素的梯度值。这包括重新计算像素的梯度幅值、梯度方向等。

- 2、根据新的梯度信息计算每个像素的前向能量。这可以使用不同的能量函数，例如基于梯度幅值的能量函数或其他更复杂的能量函数。

- 3、在计算 seam 时，使用前向能量作为优化目标，选择能量最小的路径进行 seam 删除。

通过使用前向能量，seam carving 算法能够更准确地反映像素重排引入的变化，以保持图像的结构和连续性。它能够更好地处理图像中的边缘、纹理和细节，并在图像重塑过程中提供更高的质量和保真度。

3.1.7 利用双向 Seam Carving 进行的改进

双向 Seam Carving 的基本思想是同时进行从左到右和从上到

下两个方向的能量最小路径计算。它在每个方向上计算能量图，并利用动态规划的方法确定最小能量路径。然后，从左到右和从上到下的路径分别进行标记。

接下来，根据标记的路径，选择在两个方向上具有最小能量的路径，以确保重要的结构被保留。在这个过程中，如果在某个位置上存在多个路径，可以采用不同的策略，例如随机选择路径或根据一些优化准则选择路径。

最后，选择合并这两个路径，将图像裁剪或缩放到所需的大小。这样可以更好地平衡图像中的结构保持和内容丢失，从而得到更好的结果。

双向 Seam Carving 的优点在于它可以更好地保护图像中的重要结构，并减少图像的畸变。它可以避免在单一方向上过度集中于能量最小的路径，从而提供更好的视觉效果和图像质量。然而，双向 Seam Carving 也需要更多的计算和处理，因为需要同时计算两个方向上的能量最小路径。

总而言之，双向 Seam Carving 是一种改进的 Seam Carving 算法，通过考虑两个方向上的能量最小路径来保护图像中的重要结构，以获得更好的结果。它是对传统 Seam Carving 的扩展和改进，用于图像缩放和裁剪。

3.1.8 利用多尺度的 seamcarving 进行的改进

多尺度改进的 Seam Carving 使用了不同的图像尺度来提升算法的性能。它的思想是，在多个尺度下进行 Seam Carving 操作，而不仅仅是在原始尺度上进行操作。

首先，我们会对原始图像创建一个金字塔结构，就像一系列的图像缩放版本。这些缩放版本的图像有不同的尺寸，其中一些较小，一些较大。通过这个金字塔，我们可以同时观察图像的不同细节层次。

然后，在每个尺度下，我们分别应用传统的 Seam Carving 算法。也就是说，我们在每个尺度上计算图像的能量，并找到能量最小的路径。这样，我们就可以获得每个尺度下的最优路径，以确定在该尺度上应该进行哪些裁剪或缩放操作。

最后，我们将多个尺度下的路径进行合并，以得到最终的裁剪或缩放结果。通过融合多个尺度的路径，我们可以更全面地考虑图像的不同层次的结构和内容，从而得到更好的结果。

换句话说，多尺度改进的 Seam Carving 利用了不同尺度下的信息，帮助我们更全面地理解图像的特征和内容。通过在多个尺度上进行操作，它可以处理更复杂的图像，并更好地保留图像中的重要结构。

3.1.9 利用区域感知的 Seam Carving

区域感知 Seam Carving 是一种改进的 Seam Carving 算法，旨在更好地保护图像中的重要区域，以获得更具视觉感知性的结果。

传统的 Seam Carving 算法仅基于图像的能量分布来确定路径并进行裁剪，而没有明确考虑图像中的区域重要性。这可能导致在裁剪过程中损坏关键的图像结构或重要的目标。

区域感知 Seam Carving 通过引入区域重要性的概念，考虑图像中不同区域的相对重要程度，从而更好地保护重要的区域。它的基本思想是将图像分割成不同的区域，并对每个区域赋予相应的重要性权重。

在区域感知 Seam Carving 中，重要性权重可以根据多种因素来确定，如颜色对比度、纹理信息、目标检测结果等。这些因素可以帮助识别出图像中的关键区域，如人脸、主要物体或感兴趣的细节。

一旦确定了区域的重要性权重，算法将在裁剪路径选择过程中考虑这些权重。它会倾向于选择穿过相对不重要的区域的路径，以最小化对重要区域的影响。这样可以确保在裁剪过程中保护图像中的关键部分，并减少对视觉感知性的破坏。

通过引入区域感知的重要性权重，区域感知 Seam Carving 可以更好地保护图像中的重要区域，提高结果的质量和视觉感知性。它在图像编辑、图像压缩等领域具有广泛的应用，特别适用于需要保留重要细节和结构的图像操作。

总而言之，区域感知 Seam Carving 是一种改进的 Seam Carving 算法，通过考虑图像中不同区域的重要性，以保护重要区域并获得更具视觉感知性的结果。

3.1.8 算法限制分析

Seam carving 算法的局限性在于调整图像大小时会遇到一些限制。当进行过度的 seam 删除时，特别是在敏感区域如人脸上，会导致图像产生明显的变形，看起来像是毕加索风格的特征。这是因为过度的 seam 删除会改变图像的结构和比例，对人脸等重要特征造成损害。

另一方面，过度的 seam 插入也会导致可见的拉伸伪影。当我们尝试通过插入 seam 来增大图像时，会对图像进行扩展。然而，过度的插入会导致图像中的元素被拉伸，失去原始比例和细节，从而产生明显的伪影。

因此，在使用 seam carving 算法时，需要谨慎调整调整的程

度，避免过度删除或插入 seam，以保持图像的质量和可视化效果。

3.1.9 利用 grabcut 制作遮罩优化实现过程

GrabCut 是一种用于图像分割的算法，它通过交互式方式将图像分为前景和背景两个部分。该算法最初由 Rother 等人于 2004 年提出，它结合了图像分割和图割算法的思想。

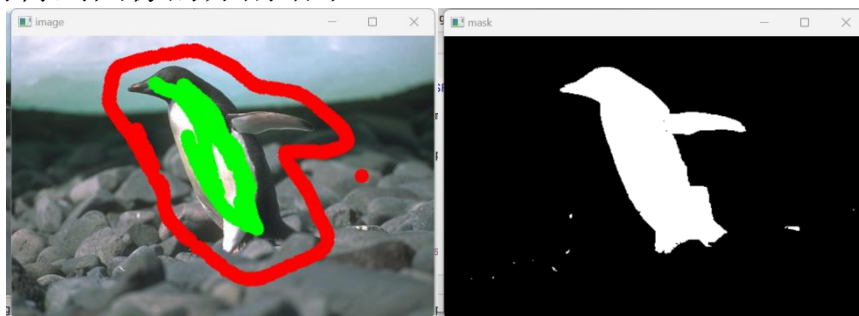
GrabCut 算法的基本思想是通过用户提供的初始矩形框来定义前景区域和背景区域，并根据这些区域的颜色、纹理和边界等特征进行图像分割。算法的核心步骤如下：

初始化：用户通过绘制一个矩形框来定义初始的前景和背景区域。

高斯混合模型（GMM）建模：GrabCut 使用 GMM 对图像中的像素进行建模。它将像素分为前景和背景两个混合成分，并通过迭代的方法估计每个像素属于前景或背景的概率。

迭代优化：通过迭代的方式，算法不断调整前景和背景的概率分布，直到达到收敛条件。在每一次迭代中，算法会通过最大似然估计来更新 GMM 参数，并利用图割算法计算每个像素属于前景或背景的概率。

分割结果：最终，通过对像素的概率进行阈值处理，将概率高于阈值的像素标记为前景，概率低于阈值的像素标记为背景，从而得到图像的分割结果。



GrabCut 算法的优点是它可以在用户交互的基础上进行图像分割，用户通过提供初始的前景和背景区域来指导算法的分割过程。同时，算法也能够自动学习图像中前景和背景的特征，并根据这些特征进行分割。然而，GrabCut 算法对于复杂的图像和复杂的前景背景边界可能表现不佳，需要额外的后期处理来改善分割结果。

3.1.8 算法优化

1. **能量计算的优化：**在 seam carving 中，传统的能量计算方法是基于像素的梯度值或局部变化。可以使用快速的滤波器算法（如 Sobel 算子）来近似计算像素梯度，从而减少计算复杂度。
2. **动态规划表格的存储优化：**

在 **Seam Carving** 算法中，计算每一行的能量和路径信息时只需要保存上一行的信息，这是因为在计算当前行的能量和路径时，只需要依赖于上一行的信息，而不需要依赖于更早的行。

Seam Carving 算法的路径计算是一个动态规划过程，它通过累积能量值来选择最小能量的路径。在计算当前行的能量时，我们只关心上一行相邻像素的能量值和路径信息，而不需要关注更早的行。

这是因为在路径计算过程中，每个像素的能量值都是根据其本身的能量和相邻像素的能量值计算得出的。在计算当前行的能量时，我们只需要关注上一行相邻像素的能量值，而不需要关心更早的行的能量值。因此，我们只需要保存上一行的信息即可进行当前行的能量计算和路径选择。

通过只保存上一行的信息，可以大大减少内存的使用，特别是在处理大尺寸图像时。因为我们不需要同时保存所有行的信息，只需按行依次计算即可。这种逐行计算和更新的方式大大降低了存储开销，并且在计算过程中仍能准确地找到能量最小的路径。

因此，在 **Seam Carving** 算法中，通过只保存上一行的信息，可以在不影响路径计算结果的前提下，显著减少存储需求，并提高算法的效率。存储优化的方法在计算每一行的能量和路径信息时只保存上一行的信息，这会影响回溯的过程。

在传统的 **Seam Carving** 算法中，为了获取最小能量路径，需要从最后一行开始回溯到第一行，沿着能量最小的路径进行选择。在回溯过程中，需要使用完整的动态规划表格来获取路径信息。

然而，如果使用存储优化的方法仅保存上一行的信息，那么在回溯过程中，我们无法直接访问完整的动态规划表格。这意味着在回溯过程中，我们需要重新计算之前行的能量和路径信息，以便找到正确的路径。

具体而言，在回溯过程中，需要从最后一行的能量最小点开始，沿着能量最小的路径向上回溯，并在每一行重新计算能量和路径信息，直到回溯到第一行为止。这样可以确保获取完整的能量最小路径。

虽然回溯过程中需要重新计算之前行的能量和路径信息，会增加一定的计算开销，但由于存储优化方法可以大大减少内存使用，因此整体上的性能仍然会有所提升。

因此，在使用存储优化的方法时，回溯过程需要重新计算能量和路径信息，以获取完整的能量最小路径。这是在存储优化和节省内存的同时，对回溯过程进行的一种折衷。

3. **递推关系的优化：**优化递推关系的计算过程可以减少不必要的重复计算。可以将能量值存储在表格中，然后在递推关系中直接引用已经计算的能量值，而不是重新计算。
4. **并行计算的优化：**由于 seam carving 的每个像素都可以独立计算能量值和选择最优路径，可以考虑使用并行计算的技术来加速算法。通过并行计算，可以同时处理多个像素，提高算法的执行效率。
5. **快速近似算法：**如果需要快速计算近似的 seam carving 结果，可以使用一些近似算法来加速计算。例如，使用近似的能量计算方法或近似的优化策略，可以在牺牲一定质量的情况下获得更快的执行速度。

3.2 代码步骤分析

3.2.1 能量计算

第一步是计算像素的能量，即其重要性的度量。能量越高，像素作为一条 seam 的一部分的可能性越小。

使用一阶梯度：

法一：

利用三个矩阵是用于计算能量图的梯度的核（Kernel）。它们分别表示了水平方向和垂直方向上计算像素梯度的权重。

`self.kernel_x` 是一个 3x3 的矩阵，用于在水平方向上计算像素梯度。它的值表示了每个像素与其相邻像素之间的权重关系，其中 `[-1, 0, 1]` 表示水平方向上的差分。

`self.kernel_y_left` 是一个 3x3 的矩阵，用于在左斜方向（从左下到右上）上计算像素梯度。它的值表示了每个像素与其相邻像素之间的权重关系，其中 `[0, 0, 1]` 表示左斜方向上的差分。

`self.kernel_y_right` 是一个 3x3 的矩阵，用于在右斜方向（从右上到左下）上计算像素梯度。它的值表示了每个像素与其相邻像素之间的权重关系，其中 `[1, 0, 0]` 表示右斜方向上的差分。

这些核矩阵被用于计算能量图中每个像素的梯度，从而反映图像中的边缘和细节信息。在累积能量映射的计算过程中，这些梯度信息被用于考虑像素与其相邻像素之间的能量变化，从而得到更准确的能量图。

法二：

还有一种常见方法是使用 Sobel 算子。Sobel 算子是一种基于离散差分的算子，用于检测图像中的边缘。它分别在水平和垂直方向上对图像进行卷积操作，然后通过求平方和再开方的方式得到像素的梯度能量。

具体而言，计算一阶梯度能量的步骤如下：

- 1、将输入图像转换为灰度图像，以便于处理。
- 2、使用 Sobel 算子对灰度图像进行水平和垂直方向上的卷积操作，得到梯度图像。
- 3、对水平和垂直方向的梯度进行平方和计算。
- 4、将平方和的结果相加，并进行开方操作，得到最终的一阶梯度能量图。

使用双梯度能量函数：

双梯度能量函数是一种用于计算像素能量的方法。它基于像素周围颜色的梯度变化来评估像素的重要性。具体来说，双梯度能量函数通过计算像素的横向和纵向梯度的平方和来确定像素的能量。

对于每个像素，双梯度能量函数分别计算其横向和纵向梯度。横向梯度是指像素左右相邻像素颜色之间的差异，纵向梯度是指像素上下相邻像素颜色之间的差异。然后，将横向和纵向梯度的平方和作为该像素的能量值。

通过使用双梯度能量函数，我们可以获得图像中每个像素的能量分布，从而确定哪些像素在图像中具有较高的能量，表示它们对图像结构的重要性。在 seam carving 中，像素能量用于确定应该移除的 seam，以保留图像中最重要的内容。

需要注意的是，双梯度能量函数考虑了像素重排后的变化，因此它比简单的梯度能量函数更能有效地捕捉到图像中的结构和边缘信息。

能量函数对图像中颜色梯度急剧变化的区域。seam-carving 技术避免删除这些高能量的像素。双梯度能量函数的计算公式如下：

$$\text{能量}(x, y) = \nabla x^2(x, y) + \nabla y^2(x, y)$$

其中，x 梯度的平方 $\nabla x^2(x, y) = R_x^2(x, y) + G_x^2(x, y) + B_x^2(x, y)$ 。

中心差分 $\nabla R(x, y)$ 、 $\nabla G(x, y)$ 和 $\nabla B(x, y)$ 是像素 $(x+1, y)$ 与像素 $(x-1, y)$ 之间红色、绿色和蓝色分量的绝对差值。y 梯度的平方 $\nabla y^2(x, y)$ 以类似的方式定义。对于图像边界上的像素，通过将最左列和最右列定义为相邻列，将最上行和最下行定义为相邻行来计算能量。例如，要计算最左列中像素 $(0, y)$ 的能量，使用它的右邻居 $(1, y)$ 和“左”邻居 $(W-1, y)$ 。

作为示例，考虑下面的 RGB 值为 3x4 的图像（每个分量是 0 到 255 之间的整数）的表格。

对于示例 1 中的非边界像素 $(1, 2)$ ，我们需要计算其能量。能量的计算涉及到像素的梯度，包括 x 梯度和 y 梯度。

首先，计算 x 梯度。我们需要计算像素(1, 2)的 R、G、B 通道与左右相邻像素之间的差值。

对于 R 通道：

$$R_x(1, 2) = |R(0, 2) - R(2, 2)| = |255 - 255| = 0$$

对于 G 通道：

$$G_x(1, 2) = |G(0, 2) - G(2, 2)| = |205 - 203| = 2$$

对于 B 通道：

$$B_x(1, 2) = |B(0, 2) - B(2, 2)| = |255 - 51| = 204$$

接下来，计算 y 梯度。我们需要计算像素(1, 2)的 R、G、B 通道与上下相邻像素之间的差值。

对于 R 通道：

$$R_y(1, 2) = |R(1, 1) - R(1, 3)| = |255 - 255| = 0$$

对于 G 通道：

$$G_y(1, 2) = |G(1, 1) - G(1, 3)| = |255 - 153| = 102$$

对于 B 通道：

$$B_y(1, 2) = |B(1, 1) - B(1, 3)| = |153 - 153| = 0$$

现在，我们可以计算能量。能量的计算使用了梯度的平方和。

对于 x 梯度的能量：

$$\nabla_x^2(1, 2) = R_x(1, 2)^2 + G_x(1, 2)^2 + B_x(1, 2)^2 = 0^2 + 2^2 + 204^2 = 41620$$

对于 y 梯度的能量：

$$\nabla_y^2(1, 2) = R_y(1, 2)^2 + G_y(1, 2)^2 + B_y(1, 2)^2 = 0^2 + 102^2 + 0^2 = 10404$$

最终，像素(1, 2)的能量为 $41620 + 10404 = 52024$ 。

对于边界像素(1, 0)，我们只需要计算其左右相邻像素之间的梯度。

两者对比

在 Seam Carving（缩减图像大小）算法中，一阶梯度和二阶梯度可以用于计算能量图，但它们在捕捉图像特征和保留重要内容方面有所区别。

1、一阶梯度能量图：

基于一阶梯度（梯度的幅度）计算能量图，反映了图像中的边缘和纹理信息。

优点：

能够捕捉到边缘和纹理等细节信息，可以在缩减图像尺寸时保留重要的图像内容。

缺点：

对于一些平滑或纹理较弱的区域，能量值可能较低，可能会导致在删除能量较低的像素时丢失一些细节。

2、二阶梯度能量图：

基于二阶梯度（梯度的变化率）计算能量图，更加关注像素之间的平滑度。

优点：

能够捕捉到平滑度信息，对于保留图像的整体结构和连续性较好。

缺点：

对于边缘和纹理等细节信息的捕捉相对较弱，可能在缩减图像尺寸时丢失一些重要细节。

选择一阶梯度还是二阶梯度能量图取决于具体应用场景和需求。一般来说，如果保留图像的边缘和纹理等细节信息更重要，则选择一阶梯度能量图；如果保留图像的整体结构和平滑度更重要，则选择二阶梯度能量图。也可以根据实际效果进行实验和比较，选择最适合的能量计算方法。

3.2.2Seam 查找

压缩和扩大操作在 Seam Carving 中确实使用了不同的能量计算方法，分别是前向计算和后向计算，以满足不同的需求。

前向计算是用来计算从图像顶部到底部的能量累积值，找到能量最低的路径。这个路径表示了缩减图像宽度时应该删除的像素列。通过选择能量累积值最小的路径，我们可以找到能量最低的区域，从而决定要保留的像素。

后向计算是用来计算从图像底部到顶部的能量累积值，找到能量最高的路径。这个路径表示了扩展图像宽度时应该插入的像素列。通过选择能量累积值最大的路径，我们可以找到能量最高的区域，从而决定要删除的像素。

综合前向计算和后向计算，我们可以根据能量最低和能量最高的路径来决定图像的缩减和扩展，实现图像的无损调整尺寸。前向计算确定要删除的路径，后向计算确定要插入的路径，两者结合起来形成了 Seam Carving 算法的核心思想。

扩大

当我们在 Seam Carving 中需要扩大图像时，后向操作用于确定插入新像素的位置。下面是一个通俗解释后向操作的过程：

首先，我们有一个能量图，其中每个像素都代表了该像素在图像中的重要性或能量值。

我们从能量图的底部开始，也就是图像的最后一行。

从最后一行开始，我们向上逐行计算能量累积值。对于每个像素，我们考虑其上方的三个相邻像素（或更少，如果当前像素在图像边缘），并选择其中能量累积值最低的路径。

我们将计算得到的能量累积值存储在一个新的累积能量图中，其中每个像素的值代表从底部到当前位置的最低能量累积值。

重复上述步骤，逐行向上计算能量累积值，直到达到图像的顶部。此时，我们得到了一张包含了最低能量累积值的累积能量图。

根据最终的累积能量图，我们可以确定应该在哪些位置插入新的像素列。插入新像素的位置取决于能量累积值的最低路径。

最后，我们在确定的位置插入新的像素列，使图像扩大。

通过后向操作，我们可以根据能量累积值找到最低能量的路径，以确定在图像扩大时应该插入的像素列，从而保持图像的重要特征和内容。

压缩

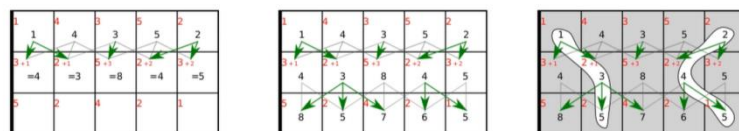
接下来的步骤是寻找总能量最小的垂直缝隙（水平缝隙的寻找方式类似）。这类似于带权有向图中的经典最短路径问题，但有三个重要的区别：

每条边的权重是基于顶点（像素）而不是边本身。

目标是找到从顶部行的任意一个像素到底部行的任意一个像素的最短路径。

有向图是无环的，从像素 (x,y) 到 $(x-1,y+1)$ ， $(x,y+1)$ ， $(x+1,y+1)$ 之间存在向下的边，假设坐标在规定范围内。

• 寻找能量的最短路径



缝隙不能环绕图像（例如，垂直缝隙不能从图像的最左列穿越到最右列）。

在上图中，我们可以看到从图像的顶部到底部的最短路径，即总能量最小的垂直缝隙。为了实现这一步骤，我们需要使用动态规划算法来搜索最短路径。

总的来说，在前向过程中，我们希望通过选择能量累积值最低的路径来压缩图像。这些路径通常经过了图像中相对较不重要的区域，例如背景或重复的纹理，因此选择能量累积值最低的路径可以保留图像中的重要特征，同时减少图像的尺寸。

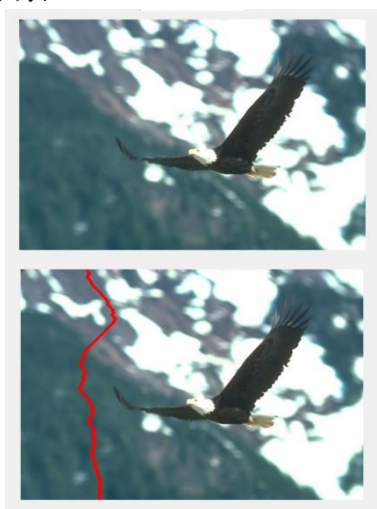
在后向过程中，我们希望通过选择能量累积值最低的路径来扩大图像。这些路径通常经过了图像中较为重要的区域，例如主要对象或感兴趣的区域，因此选择能量累积值最低的路径可以保持图像的重要特征，并在插入新像素时确保图像的连续性和一致

性。

虽然能量累积值可能是相同的，但路径的选择是根据不同的目标进行的。因此，前向过程和后向过程所对应的路径是不同的，以实现不同的图像处理效果。

3.2.3 去除接缝

Seam removal 是最后一步，它的目标是从图像中移除所有沿着垂直或水平缝隙的像素。



四、算法实现

在本文中，一共设计了两个 ui 界面。

4.1 三种方式的 ui 界面

其一是三种功能的使用，遮罩采取 grabcut 的形式完成，设计了 ui 界面集成三种功能，让交互性更好。



```
seam_carving.py
import numpy as np
import cv2

class SeamCarver:
    def __init__(self, filename, out_height, out_width, protect_mask='',
object_mask=''):
        self.filename = filename
```

```

self.out_height = out_height
self.out_width = out_width

self.in_image = cv2.imread(filename).astype(np.float64)
self.in_height, self.in_width = self.in_image.shape[: 2]

self.out_image = np.copy(self.in_image)

self.object = (object_mask != '')
if self.object:
    # read in object mask image file as np.float64 format in gray scale
    self.mask = cv2.imread(object_mask, 0).astype(np.float64)
    self.protect = False
    # image re-sizing with or without protect mask
else:
    self.protect = (protect_mask != '')
    if self.protect:
        self.mask = cv2.imread(protect_mask,
0).astype(np.float64)

        self.kernel_x = np.array([[0., 0., 0.], [-1., 0., 1.], [0., 0., 0.]],
dtype=np.float64)
        self.kernel_y_left = np.array([[0., 0., 0.], [0., 0., 1.], [0., -1., 0.]],
dtype=np.float64)
        self.kernel_y_right = np.array([[0., 0., 0.], [1., 0., 0.], [0., -1., 0.]],
dtype=np.float64)

        self.constant = 1000
print("初始化完毕")
self.start()

def start(self):
    if self.object:
        self.object_removal()
    else:
        self.seams_carving()

def seams_carving(self):
    print("seams_carving")
    delta_row, delta_col = int(self.out_height - self.in_height),
int(self.out_width - self.in_width)

    # remove column
    if delta_col < 0:
        self.seams_removal(delta_col * -1)
    # insert column
    elif delta_col > 0:
        self.seams_insertion(delta_col)

```

```

# remove row
if delta_row < 0:
    self.out_image = self.rotate_image(self.out_image, 1)
    if self.protect:
        self.mask = self.rotate_mask(self.mask, 1)
    self.seams_removal(delta_row * -1)
    self.out_image = self.rotate_image(self.out_image, 0)
# insert row
elif delta_row > 0:
    self.out_image = self.rotate_image(self.out_image, 1)
    if self.protect:
        self.mask = self.rotate_mask(self.mask, 1)
    self.seams_insertion(delta_row)
    self.out_image = self.rotate_image(self.out_image, 0)

def object_removal(self):
    print("object_removal")
    rotate = False
    object_height, object_width = self.get_object_dimension()
    if object_height < object_width:
        self.out_image = self.rotate_image(self.out_image, 1)
        self.mask = self.rotate_mask(self.mask, 1)
        rotate = True

    while len(np.where(self.mask[:, :] > 0)[0]) > 0:
        energy_map = self.calc_energy_map()
        energy_map[np.where(self.mask[:, :] > 0)] *= -self.constant
        cumulative_map = self.cumulative_map_forward(energy_map)
        seam_idx = self.find_seam(cumulative_map)
        self.delete_seam(seam_idx)
        self.delete_seam_on_mask(seam_idx)

    if not rotate:
        num_pixels = self.in_width - self.out_image.shape[1]
    else:
        num_pixels = self.in_height - self.out_image.shape[1]

    self.seams_insertion(num_pixels)
    if rotate:
        self.out_image = self.rotate_image(self.out_image, 0)

def seams_removal(self, num_pixel):
    print("seams_removal")
    if self.protect:
        for dummy in range(num_pixel):
            energy_map = self.calc_energy_map()
            energy_map[np.where(self.mask > 0)] *= self.constant
            cumulative_map = self.cumulative_map_forward(energy_map)
            seam_idx = self.find_seam(cumulative_map)

```

```

        self.delete_seam(seam_idx)
        self.delete_seam_on_mask(seam_idx)
    else:
        for dummy in range(num_pixel):
            energy_map = self.calc_energy_map()
            cumulative_map = self.cumulative_map_forward(energy_map)
            seam_idx = self.find_seam(cumulative_map)
            self.delete_seam(seam_idx)

def seams_insertion(self, num_pixel):
    print("seams_insertion")
    if self.protect:
        temp_image = np.copy(self.out_image)
        temp_mask = np.copy(self.mask)
        seams_record = []

        for dummy in range(num_pixel):
            energy_map = self.calc_energy_map()
            energy_map[np.where(self.mask[:, :] > 0)] *= self.constant
            cumulative_map = self.cumulative_map_backward(energy_map)
            seam_idx = self.find_seam(cumulative_map)
            seams_record.append(seam_idx)
            self.delete_seam(seam_idx)
            self.delete_seam_on_mask(seam_idx)

        self.out_image = np.copy(temp_image)
        self.mask = np.copy(temp_mask)
        n = len(seams_record)
        for dummy in range(n):
            seam = seams_record.pop(0)
            self.add_seam(seam)
            self.add_seam_on_mask(seam)
            seams_record = self.update_seams(seams_record, seam)
    else:
        temp_image = np.copy(self.out_image)
        seams_record = []

        for dummy in range(num_pixel):
            energy_map = self.calc_energy_map()
            cumulative_map = self.cumulative_map_backward(energy_map)
            seam_idx = self.find_seam(cumulative_map)
            seams_record.append(seam_idx)
            self.delete_seam(seam_idx)

        self.out_image = np.copy(temp_image)
        n = len(seams_record)
        for dummy in range(n):
            seam = seams_record.pop(0)
            self.add_seam(seam)
            seams_record = self.update_seams(seams_record, seam)

```

```

def calc_energy_map(self):
    b, g, r = cv2.split(self.out_image)
    b_energy = np.absolute(cv2.Scharr(b, -1, 1, 0)) + np.absolute(cv2.Scharr(b,
-1, 0, 1))
    g_energy = np.absolute(cv2.Scharr(g, -1, 1, 0)) + np.absolute(cv2.Scharr(g,
-1, 0, 1))
    r_energy = np.absolute(cv2.Scharr(r, -1, 1, 0)) + np.absolute(cv2.Scharr(r,
-1, 0, 1))
    return b_energy + g_energy + r_energy

```

```

def cumulative_map_backward(self, energy_map):
    m, n = energy_map.shape
    output = np.copy(energy_map)
    for row in range(1, m):
        for col in range(n):
            output[row, col] = \
                energy_map[row, col] + np.amin(output[row - 1, max(col - 1, 0):
min(col + 2, n - 1)])
    return output

```

```

def cumulative_map_forward(self, energy_map):
    matrix_x = self.calc_neighbor_matrix(self.kernel_x)
    matrix_y_left = self.calc_neighbor_matrix(self.kernel_y_left)
    matrix_y_right = self.calc_neighbor_matrix(self.kernel_y_right)

    m, n = energy_map.shape
    output = np.copy(energy_map)
    for row in range(1, m):
        for col in range(n):
            if col == 0:
                e_right = output[row - 1, col + 1] + matrix_x[row - 1, col +
1] + matrix_y_right[row - 1, col + 1]
                e_up = output[row - 1, col] + matrix_x[row - 1, col]
                output[row, col] = energy_map[row, col] + min(e_right, e_up)
            elif col == n - 1:
                e_left = output[row - 1, col - 1] + matrix_x[row - 1, col - 1]
+ matrix_y_left[row - 1, col - 1]
                e_up = output[row - 1, col] + matrix_x[row - 1, col]
                output[row, col] = energy_map[row, col] + min(e_left, e_up)
            else:
                e_left = output[row - 1, col - 1] + matrix_x[row - 1, col - 1]
+ matrix_y_left[row - 1, col - 1]
                e_right = output[row - 1, col + 1] + matrix_x[row - 1, col +
1] + matrix_y_right[row - 1, col + 1]
                e_up = output[row - 1, col] + matrix_x[row - 1, col]
                output[row, col] = energy_map[row, col] + min(e_left, e_right,
e_up)

```



```

        return output

def calc_neighbor_matrix(self, kernel):
    b, g, r = cv2.split(self.out_image)
    output = np.absolute(cv2.filter2D(b, -1, kernel=kernel)) + \
        np.absolute(cv2.filter2D(g, -1, kernel=kernel)) + \
        np.absolute(cv2.filter2D(r, -1, kernel=kernel))
    return output

def find_seam(self, cumulative_map):
    m, n = cumulative_map.shape
    output = np.zeros((m,), dtype=np.uint32)
    output[-1] = np.argmin(cumulative_map[-1])
    for row in range(m - 2, -1, -1):
        prv_x = output[row + 1]
        if prv_x == 0:
            output[row] = np.argmin(cumulative_map[row, : 2])
        else:
            output[row] = np.argmin(cumulative_map[row, prv_x - 1: min(prv_x
+ 2, n - 1)]) + prv_x - 1
    return output

def delete_seam(self, seam_idx):
    m, n = self.out_image.shape[: 2]
    output = np.zeros((m, n - 1, 3))
    for row in range(m):
        col = seam_idx[row]
        output[row, :, 0] = np.delete(self.out_image[row, :, 0], [col])
        output[row, :, 1] = np.delete(self.out_image[row, :, 1], [col])
        output[row, :, 2] = np.delete(self.out_image[row, :, 2], [col])
    self.out_image = np.copy(output)

def add_seam(self, seam_idx):
    m, n = self.out_image.shape[: 2]
    output = np.zeros((m, n + 1, 3))
    for row in range(m):
        col = seam_idx[row]
        for ch in range(3):
            if col == 0:
                p = np.average(self.out_image[row, col: col + 2, ch])
                output[row, col, ch] = self.out_image[row, col, ch]
                output[row, col + 1, ch] = p
                output[row, col + 1:, ch] = self.out_image[row, col:, ch]
            else:
                p = np.average(self.out_image[row, col - 1: col + 1, ch])
                output[row, : col, ch] = self.out_image[row, : col, ch]
                output[row, col, ch] = p

```

```

        output[row, col + 1:, ch] = self.out_image[row, col:, ch]
self.out_image = np.copy(output)

def update_seams(self, remaining_seams, current_seam):
    output = []
    for seam in remaining_seams:
        seam[np.where(seam >= current_seam)] += 2
        output.append(seam)
    return output

def rotate_image(self, image, ccw):
    m, n, ch = image.shape
    output = np.zeros((n, m, ch))
    if ccw:
        image_flip = np.fliplr(image)
        for c in range(ch):
            for row in range(m):
                output[:, row, c] = image_flip[row, :, c]
    else:
        for c in range(ch):
            for row in range(m):
                output[:, m - 1 - row, c] = image[row, :, c]
    return output

def rotate_mask(self, mask, ccw):
    m, n = mask.shape
    output = np.zeros((n, m))
    if ccw > 0:
        image_flip = np.fliplr(mask)
        for row in range(m):
            output[:, row] = image_flip[row, :]
    else:
        for row in range(m):
            output[:, m - 1 - row] = mask[row, :]
    return output

def delete_seam_on_mask(self, seam_idx):
    m, n = self.mask.shape
    output = np.zeros((m, n - 1))
    for row in range(m):
        col = seam_idx[row]
        output[row, :] = np.delete(self.mask[row, :], [col])
    self.mask = np.copy(output)

def add_seam_on_mask(self, seam_idx):
    m, n = self.mask.shape

```

```

        output = np.zeros((m, n + 1))
        for row in range(m):
            col = seam_idx[row]
            if col == 0:
                p = np.average(self.mask[row, col: col + 2])
                output[row, col] = self.mask[row, col]
                output[row, col + 1] = p
                output[row, col + 1: ] = self.mask[row, col: ]
            else:
                p = np.average(self.mask[row, col - 1: col + 1])
                output[row, : col] = self.mask[row, : col]
                output[row, col] = p
                output[row, col + 1: ] = self.mask[row, col: ]
        self.mask = np.copy(output)

    def get_object_dimension(self):
        rows, cols = np.where(self.mask > 0)
        height = np.amax(rows) - np.amin(rows) + 1
        width = np.amax(cols) - np.amin(cols) + 1
        return height, width

    def save_result(self, filename):
        cv2.imwrite(filename, self.out_image.astype(np.uint8))
ui.py:

import tkinter as tk
from tkinter import filedialog

import cv2
import numpy as np
from PIL import Image, ImageTk
from seam_carving import SeamCarver
import os
folder_in = 'in'
folder_out = 'out'
new_height = 479
new_width = 319

def image_resize_without_mask(filename_input, filename_output, new_height,
new_width):
    obj = SeamCarver(filename_input, new_height, new_width)
    obj.save_result(filename_output)

def image_resize_with_mask(filename_input, filename_output, new_height, new_width,
filename_mask):
    obj = SeamCarver(filename_input, new_height, new_width,
protect_mask=filename_mask)
    obj.save_result(filename_output)

```

```

def invert_mask(mask):
    inverted_mask = cv2.bitwise_not(mask)
    return inverted_mask

def object_removal(filename_input, filename_output, filename_mask):
    mask = cv2.imread(filename_mask, 0)
    inverted_mask = invert_mask(mask)

    inverted_mask_filename = 'inverted_mask.png'
    cv2.imwrite(inverted_mask_filename, inverted_mask)

    obj = SeamCarver(filename_input, 0, 0, object_mask=filename_mask)
    obj.save_result(filename_output)

    os.remove(inverted_mask_filename)

class ImageProcessingApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Image Processing App")
        self.filename_output = 'image_result.png'
        self.new_height = tk.StringVar()
        self.new_width = tk.StringVar()

        self.grabcut_button = tk.Button(self.root, text="运行 GrabCut",
command=self.run_grabcut)
        self.grabcut_button.pack()
        self.operation_complete_label = tk.Label(self.root, text="")
        self.operation_complete_label.pack()

        # 创建标签用于显示原图大小
        self.image_size_label = tk.Label(self.root)
        self.image_size_label.pack()

        height_label = tk.Label(self.root, text="图像新高度")
        height_label.pack()
        self.height_entry = tk.Entry(self.root, textvariable=new_height)
        self.height_entry.pack()

        width_label = tk.Label(self.root, text="图像新宽度")
        width_label.pack()
        self.width_entry = tk.Entry(self.root, textvariable=new_width)
        self.width_entry.pack()

        # 创建图像显示区域
        self.original_image_label = tk.Label(self.root)
        self.original_image_label.pack(side=tk.LEFT)
        self.processed_image_label = tk.Label(self.root)
        self.processed_image_label.pack(side=tk.RIGHT)

```

```

        # 创建按钮和下拉框
        self.import_image_button = tk.Button(self.root, text="导入原图",
command=self.import_image)
        self.import_image_button.pack()
        self.import_mask_button = tk.Button(self.root, text="导入遮罩",
command=self.import_mask)
        self.import_mask_button.pack()

        self.operations = ["调整图像大小（无遮罩）", "调整图像大小（带遮罩）", "
移除对象"]
        self.selected_operation = tk.StringVar()
        self.operation_dropdown = tk.OptionMenu(self.root,
self.selected_operation, *self.operations)
        self.operation_dropdown.pack()

        self.process_image_button = tk.Button(self.root, text="处理图像",
command=self.process_image)
        self.process_image_button.pack()

    def import_image(self):
        filename_input = filedialog.askopenfilename(title="Select Input Image")
        self.input_image = os.path.join(folder_in, "images", filename_input)
        # 加载结果图片
        result_image = Image.open(self.input_image)
        # 将结果图片显示在标签上
        image_tk = ImageTk.PhotoImage(result_image)
        self.original_image_label.configure(image=image_tk)
        self.original_image_label.image = image_tk # 保持对图片对象的引用
        # 显示原图大小
        image_width, image_height = result_image.size
        self.image_size_label.config(text=f"原图大小: {image_width} x
{image_height}")
        # 刷新界面
        self.root.update()

    def import_mask(self):
        filename_mask = filedialog.askopenfilename(title="Select Mask Image")
        self.input_mask = os.path.join(folder_in, "masks", filename_mask)

    def process_image(self):
        selected_operation = self.selected_operation.get()
        if selected_operation == "调整图像大小（无遮罩）":
            self.new_height = int(self.height_entry.get())
            self.new_width = int(self.width_entry.get())
            image_resize_without_mask(self.input_image, self.filename_output,
self.new_height, self.new_width)
            self.operation_complete_label.config(text="调整图像大小（无遮罩）完
成")
        elif selected_operation == "调整图像大小（带遮罩）":
            self.new_height = int(self.height_entry.get())
            self.new_width = int(self.width_entry.get())

```

```

        image_resize_with_mask(self.input_image, self.filename_output,
                                self.new_height, self.new_width,
                                self.input_mask)
        self.operation_complete_label.config(text="调整图像大小（带遮罩）完成")
    elif selected_operation == "移除对象":
        object_removal(self.input_image, self.filename_output,
                        self.input_mask)
        self.operation_complete_label.config(text="移除对象完成")

    result_image = Image.open(self.filename_output)
    # 将结果图片显示在标签上
    image_tk = ImageTk.PhotoImage(result_image)
    self.processed_image_label.configure(image=image_tk)
    self.processed_image_label.image = image_tk  # 保持对图片对象的引用
    # 刷新界面
    self.root.update()

def merge(self, mask_fore, mask_back, img):
    mask = mask_fore[:, :, 1:2] / 255 + mask_back[:, :, 2:3] / 255
    if np.max(np.max(np.max(mask))) > 1:
        return False, img
    mask = mask.astype('uint8')
    return True, mask_fore + mask_back + img * (1 - mask)

def draw_circle(self, event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        self.drawing = True
        self.ix, self.iy = x, y
    if event == cv2.EVENT_RBUTTONDOWN:
        self.mode = not self.mode

    elif event == cv2.EVENT_MOUSEMOVE:
        if self.drawing is True:
            if self.mode is True:
                cv2.circle(self.mask_fore, (x, y), self.r, (0, 255, 0), -1)
            else:
                cv2.circle(self.mask_back, (x, y), self.r, (0, 0, 255), -1)

    elif event == cv2.EVENT_LBUTTONUP:
        self.drawing = False
        if self.mode is True:
            cv2.circle(self.mask_fore, (x, y), self.r, (0, 255, 0), -1)
        else:
            cv2.circle(self.mask_back, (x, y), self.r, (0, 0, 255), -1)

def run_grabcut(self):
    img = cv2.imread(self.input_image)

    # GrabCut 所需内部调用的参数
    bgdModel = np.zeros((1, 65), np.float64)

```



```

fgdModel = np.zeros((1, 65), np.float64)
iteration = 10

# 初始化前景和背景的 mask
mask_fore = np.zeros_like(img)
mask_back = np.zeros_like(img)
drawing = False
mode = True
ix, iy = -1, -1
r = 8

def draw_circle(event, x, y, flags, param):
    nonlocal ix, iy, drawing, mode, mask_fore, mask_back
    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y
    if event == cv2.EVENT_RBUTTONDOWN:
        mode = not mode

    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing is True:
            if mode is True:
                cv2.circle(mask_fore, (x, y), r, (0, 255, 0), -1)
            else:
                cv2.circle(mask_back, (x, y), r, (0, 0, 255), -1)

    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False
        if mode is True:
            cv2.circle(mask_fore, (x, y), r, (0, 255, 0), -1)
        else:
            cv2.circle(mask_back, (x, y), r, (0, 0, 255), -1)

cv2.namedWindow('image')
cv2.setMouseCallback('image', draw_circle)

while True:
    valid, masked = self.merge(mask_fore, mask_back, img)
    if not valid:
        print('Foreground has interaction with background! Please
restart!')
        mask_fore = np.zeros_like(img)
        mask_back = np.zeros_like(img)
        cv2.imshow('image', masked)
        k = cv2.waitKey(1) & 0xFF

    # change mode
    if k == ord('m') or k == ord('l'):
        mode = not mode
        if mode:
            print('Drawing foreground')

```

```

        else:
            print('Drawing background')
        # clear all mask
    elif k == ord('r') or k == ord('2'):
        print('Restart')
        mask_fore = np.zeros_like(img)
        mask_back = np.zeros_like(img)
        # clear foreground mask
    elif k == ord('4'):
        print('Reselect foreground')
        mask_fore = np.zeros_like(img)
        # clear background mask
    elif k == ord('5'):
        print('Reselect background')
        mask_back = np.zeros_like(img)
        # run grabcut algorithm
    elif k == ord('c') or k == ord('3') or k == 13:
        print('Cutting foreground from the picture')
        mask_global = np.zeros(img.shape[:2], np.uint8) + 2
        mask_global[mask_fore[:, :, 1] == 255] = 1
        mask_global[mask_back[:, :, 2] == 255] = 0
        mask_global, bgdModel, fgdModel = cv2.grabCut(
            img, mask_global, None, bgdModel, fgdModel, iteration,
            cv2.GC_INIT_WITH_MASK)
        mask_global = np.where((mask_global == 2) | (mask_global == 0), 0,
            1).astype('uint8')

        # 将 GrabCut 的结果转换为遮罩格式
        mask_output = mask_global * 255

        # 显示遮罩结果
        cv2.imshow('mask', mask_output)
        cv2.waitKey(0)
        # 将遮罩保存为图像文件
        mask_filename = 'mask.png'
        cv2.imwrite(mask_filename, mask_output)

        # 这里可以对遮罩进行进一步处理或者保存, 根据你的需求

        cv2.destroyAllWindows()
        cv2.imwrite(self.filename_output, img)
        self.input_mask = mask_filename
        break

    elif k == 27:
        break
cv2.destroyAllWindows()

def run(self):
    self.root.mainloop()

```

```

if __name__ == "__main__":
    root = tk.Tk()
    app = ImageProcessingApp(root)
    app.run()

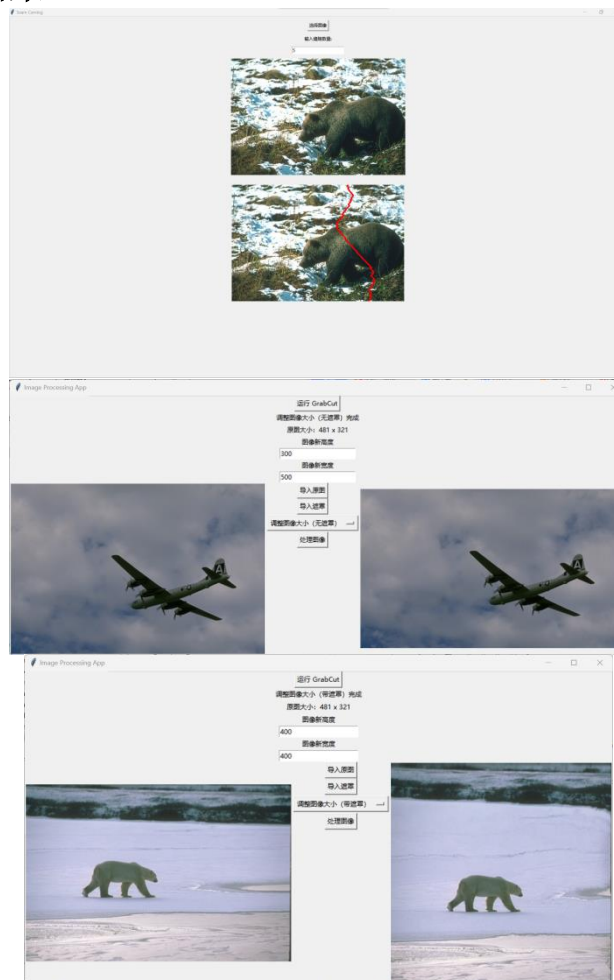
```

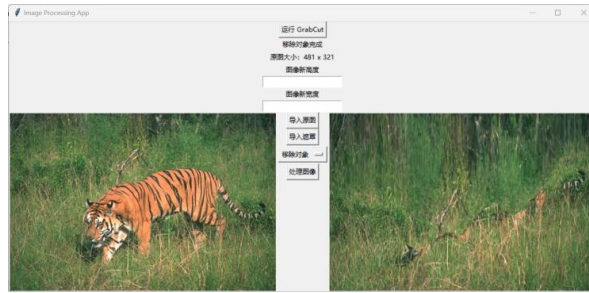
4.2 动态展示分割过程



实验结果：(截图)

1、程序运行结果：





2、时间复杂度分析：

保持图片宽度不变，通过改变图像高度来反应输入规模的变化，输入不同规模的图像运行，并计算其运行时间，经过上文对于时间复杂度的分析，我们可以认为如果运行结果的数据规模和时间接近于线性关系，那么我们的分析是合理的。

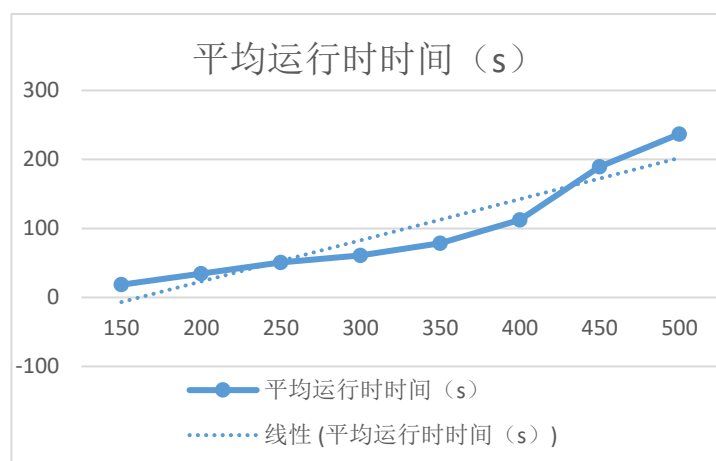


3、结果分析：

多次试验取平均值减小误差后的运行结果如下：

输入规模（高度）	平均运行时时间（s）
150	18.49
200	34.36
250	50.54
300	61.08
350	78.54
400	112.42
450	189.36
500	236.95

绘制时间复杂度曲线：



可以看出这个代码和我们之前的实验运行时间相比还是大大拉长了，本次实验的代码是较为复杂的。至此我们还可以对动态规划这一知识点进行总结：

优点：动态规划通过将问题分解为子问题，并使用子问题的解来构建整体解决方案，可以找到全局最优解。它可以处理具有重叠子问题和最优子结构特性的问题，并通过存储和重复使用子问题的解来提高效率。

缺点：动态规划算法的实现可能较为复杂，需要设计合适的状态转移方程和记忆化存储。对于大规模问题，动态规划的计算开销可能会很大。

对于缝隙修剪问题来说，动态规划方法通常能够找到全局最优的缝隙路径，但需要更多的计算开销。有的文章中提及的贪心算法虽然简单快速，但可能无法保证找到最优解。

选择使用贪心算法还是动态规划方法取决于问题的规模、时间要求和对解决方案准确性的要求。如果对结果的准确性要求较高，或者问题规模较小且允许一定的计算开销，动态规划是一个更好的选择。如果问题规模较大且时间要求较高，贪心算法可能是一个更合适的选择，尽管它可能无法找到最优解。

实验心得：

本实验旨在使用动态规划算法实现 Seam Carving 技术，通过图像的压缩和扩大来调整图像的尺寸。在实验中，我研究并完成了利用动态规划进行 seamcarving 的几种基本实现算法，并相应做出了可视化 gui 界面呈现实验结果。

在实验中，我遇到了一些挑战和学习点。首先，理解动态规划算法的原理和应用是关键。动态规划能够高效地解决最优化问题，但其实现需要合理的状态定义和状态转移方程。通过仔细研究算法思想和参考相关文献，我成功地将动态规划应用于 Seam Carving 技术中。

通过完成本实验，我对 Seam Carving 技术和动态规划算法有了更深入的理解。Seam Carving 是一种非常有用和强大的图像处理技术，可以在不改变图像内容的情况下调整图像的尺寸。动态规划作为解决最优化问题的有效算法，能够高效地找到最小能量路径，为图像的压缩和扩大提供了基础。

在实验中，我深入学习如何实现动态规划算法，并将其应用于 Seam Carving 技术中。我深入理解了累积能量图的构建过程以及路径查找的方法。同时，我意识到能量计算对于结果的影响，合理选择能量计算方法对于保留图像的重要特征至关重要。

此外，我还学会了处理图像数据和像素操作的技巧。通过插入和删除像素，我成功地调整了图像的尺寸，并保持了图像的重要内容。这为我在后续的图像处理任务中提供了更多的思路和技能。

这次实验也是我第一次使用 python 制作 gui 界面，在制作过程中我收获丰富，学习到了很多关键点，同时也体会到了制作 gui 界面来可视化结果的实用性。

参考文献

<https://zhuanlan.zhihu.com/p/65339171>
<https://www.win.tue.nl/~wstahw/edu/2IV05/seamcarving.pdf>
<https://github.com/vivianhylee/seam-carving>
https://blog.csdn.net/wsp_1138886114/article/details/104076134
https://www.bilibili.com/video/BV1hA411L7Bp/?spm_id_from=333.880.my_history.page.click
<https://zhuanlan.zhihu.com/p/65339171>